WEEK 06

ARM Core Architecture and Memory

Opening the CPU. Registers, the memory map, the stack, and how a line of C turns into three instructions

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 4, printed pages 91–107.
Lab manual: Lab 10 (stack and heap on the STM32F303), Section 10.2.2.

1 What we are actually after this week

For five weeks we have treated the CPU as a box that runs C. This week we open it, because in week 7 we need to explain what "the processor saves the program's current information" actually means when an interrupt fires, and you cannot understand that without knowing which registers exist and where the stack lives.

The plan:

1. The anatomy of a CPU core: program counter, register file, ALU, control logic, and the fetch-decode-execute loop between them.
2. What an *architecture* is, and the two big commitments ARM made: **load/store** and a 32-bit **native data type**.
3. **The register file.** Sixteen registers, three of which are special, plus three system registers. This is the single most exam-worthy page in the chapter.
4. **The memory map.** A 4 GB address space carved into regions, and where your 256 KB of flash and 40 KB of SRAM actually sit inside it.
5. **Endianness.** Short, but it turns up the moment you send a multi-byte value over SPI.
6. **The stack.** How it grows, what `PUSH` and `POP` do to `SP`, and why it grows downward.
7. **Instructions.** What the categories are, how a subroutine call and return work, and why `LR` exists.

Items 3, 4, and 6 are the ones that carry into week 7 and week 10. Item 7 gets a survey rather than a full treatment, since week 10 is the assembly chapter.

2 Deep dive 1: What is inside a CPU core

Recall the block diagram of an MCU from Week 01: a core, memory, and peripherals. Zoom into the core and here is what an instruction meets.

Instruction, operation, operand

> An **instruction** is a command for the processor to execute. It consists of an **operation**, which specifies what work to do, and zero or more **operands**, which are the parameters the operation uses.

Five components:

- **Program memory** holds the instructions. The **program counter (PC)** is a register holding the address of the next instruction to execute.
- **The register file** holds temporary data values just before and just after processing. Fast to access, and *small*, holding only a handful of items.
- **The ALU** (arithmetic/logic unit) is the heart of the thing. Addition, subtraction, AND, OR, comparison. It takes operands from the register file or from inside the instruction itself.
- **Data memory** is the long-term store. Far larger, holding thousands or millions of items, and slower and more complex to reach.
- **Control logic** decodes each instruction into the control and data signals that drive everything else.

The cycle, then: read the instruction at the address in PC, decode it, and use the decoded information to decide which registers to read as sources, how to generate any other operands, whether the ALU acts (and which operation) or memory is accessed, which register receives the result, and how to update PC.

Normally PC advances to the next instruction. A **control-flow instruction** such as a branch, subroutine call, or return, *or an interrupt request*, makes control jump elsewhere. That parenthetical is the hinge of the whole chapter: from the CPU's point of view an interrupt is just another thing that changes PC.

Some cores speed this up by **pipelining**, starting the next instruction before the current one has finished.

Book board vs your board

> Both your part and the book's have a **3-stage pipeline** (fetch, decode, execute). The Cortex-M4 adds branch speculation, so it can begin fetching down a predicted path, which the M0 does not do.
>
> There is a more consequential difference in the bus structure. The Cortex-M0 uses a **von Neumann** architecture, one bus shared between instruction fetches and data accesses, so those two compete. The Cortex-M4 uses a **Harvard** architecture with separate instruction and data buses, so a fetch and a load can proceed simultaneously. That is a large part of why the M4 achieves more work per clock, quite apart from its higher clock.

3 Deep dive 2: What an architecture commits you to

Architecture

> The specification of a processor's **programmer's model**: the operations available, the registers, the memory organisation, and the behaviour visible to software. An architecture is a contract, and many different chips can implement the same one.

The Cortex-M0 implements **ARMv6-M**, a specialised, smaller profile of ARMv6. Two commitments in that contract matter enormously in practice.

3.1 Load/store architecture

Only data in registers can be processed. Data in memory cannot be operated on directly. It must be loaded into a register, processed, and perhaps stored back.

This is why the smallest useful example in the chapter is three instructions rather than one:

Listing 1: Machine language on the left, assembly on the right. Book Listing 4.1.

```
9800     LDR   R0, [SP, #0]      ; load R0 from memory at address SP+0
1900     ADDS  R0, R0, R4        ; R0 = R0 + R4
9000     STR   R0, [SP, #0]      ; store R0 back to memory at SP+0
```

Three instructions to add something to a variable in memory: load, add, store. On an architecture that permitted memory operands you might do it in one. ARM does not, and the payoff is that the hardware is simpler, which generally means faster and lower power. Recall the constraints list from Week 01: this is a deliberate trade of instruction count against silicon complexity and energy.

Beware the trap

This is why the read-modify-write pattern from Week 03 is genuinely three memory transactions and not an atomic operation. `GPIOA->ODR |= MASK(5)` compiles to load, OR, store, and an interrupt can land between the load and the store. That is not a quirk of the compiler, it is a direct consequence of load/store architecture, and it is exactly why `BSRR` exists.

3.2 The native data type is 32-bit

Native data type

The primary data type used by the ALU and registers, which the hardware supports directly and processes quickly. For Arm Cortex-M this is the **32-bit integer**, signed and unsigned.

Operations on other types can be *emulated* by multiple instructions, which takes more time. And the practical consequences are immediate and worth internalising now, because they will bite you in your project:

- A `uint32_t` load or store is one instruction, therefore atomic. A `uint64_t` is two, therefore not.
- `int` arithmetic is free. `int64_t` arithmetic costs several instructions per operation.
- Floating point on the M0 is *entirely software*, dozens to hundreds of instructions per operation.

Book board vs your board

This last point is the single biggest practical gap between the two chips. The book's Cortex-M0 has **no FPU**, so every `float` multiply is a library call. Your Cortex-M4 has a **single-precision hardware FPU**, so a `float` multiply is one instruction.

Two consequences for you. First, `float` is cheap on your board and `double` is *not*, because the F303's FPU is single precision only, so `double` falls back to software. Write `2.0f` rather than `2.0` and use `sinf()` rather than `sin()`, or you will silently invoke the software path. Second, your PID controller in the self-balancing project can reasonably use `float`, which on the book's M0 would be a bad idea.

The M4 also adds **DSP instructions**, including single-cycle multiply-accumulate and SIMD operations on packed 16-bit values. That is what makes the CMSIS-DSP filter in your Lab 07 practical.

4 Deep dive 3: The registers

Here is the page to memorise. Sixteen core registers, all 32 bits. Assemblers accept upper or lower case, so `R0` and `r0` are the same thing.

Register	Name	Purpose
R0–R12	—	General purpose, for data processing
R13	**SP**	**Stack pointer.** Points to the last item on the stack
R14	**LR**	**Link register.** Holds the return address after a Branch-and-Link
R15	**PC**	**Program counter.** Address of the next instruction

R13 is worth a second look, because there are actually two of them. The architecture provides a **Main Stack Pointer (MSP)** and a **Process Stack Pointer (PSP)**, and only one is visible as R13 at any moment. Simple applications use only MSP. A kernel or operating system may use both, typically MSP for exception handlers and kernel code, PSP for application tasks, so that a runaway task cannot corrupt the kernel's stack. Recall the RTOS discussion at the end of Week 05: this is the hardware feature that makes that separation possible.

4.1 The three special registers

PSR, the program status register, is one physical register with three different *views*, and the exam question is usually "name the views and what each shows":

View	Stands for	Contains
APSR	Application PSR	The condition code flags: **N**egative, **Z**ero, **C**arry, **V** overflow. Set by instructions based on their result
IPSR	Interrupt PSR	The exception number of the currently executing handler
EPSR	Execution PSR	Whether the CPU is in Thumb mode

PRIMASK: set it to one and the CPU ignores some classes of exception. This is the global interrupt disable, and it is the mechanism behind critical sections in Week 08.

CONTROL: one field, `SPSEL`, choosing which stack pointer is used in Thread mode. That is the MSP/PSP switch.

Beware the trap

Notice **IPSR holds the current exception number**. That is how a handler can discover which exception it is servicing, and it is also how code can ask "am I currently inside an ISR?" A nonzero IPSR means yes.

This matters practically. A function that behaves differently inside an ISR, for instance one that must not block, can check IPSR. And it explains a debugging trick: if you halt the processor and IPSR is nonzero, you are stopped inside a handler, which is often the answer to "why is my main loop not running?"

Book board vs your board

The register file is *identical* on both parts. R0–R12, SP, LR, PC, PSR with three views, PRIMASK, CONTROL. Everything in Section 3 transfers unchanged.

Your Cortex-M4 adds registers the M0 lacks: **FAULTMASK** and **BASEPRI** for finer interrupt

masking, and **S0–S31** plus `FPSCR` for the FPU. `BASEPRI` is the interesting one: instead of PRIMASK's all-or-nothing disable, it lets you block interrupts *below a given priority* while leaving urgent ones live. That is a much better tool for critical sections, and we will come back to it in Week 08.

5 Deep dive 4: The memory map

ARMv6-M has a 32-bit address space, so 2^{32} addressable locations. Each address specifies one **byte**, so memory is **byte-addressable**, and 2^{32} bytes is 4 GB of address space.

Which is a lot, for a chip with 296 KB of actual memory. So most of it is empty, and the architecture divides the space into fixed regions by purpose:

Region	Address range	Holds
Code	`0x0000_0000`–`0x1FFF_FFFF`	Program instructions
SRAM	`0x2000_0000`–`0x3FFF_FFFF`	On-chip data memory
Peripheral	`0x4000_0000`–`0x5FFF_FFFF`	On-chip peripheral registers
External RAM	`0x6000_0000`–`0x9FFF_FFFF`	Off-chip RAM
External device	`0xA000_0000`–`0xDFFF_FFFF`	Off-chip devices
Cortex-M peripherals	`0xE000_0000`–`0xE00F_FFFF`	NVIC, SysTick, debug
System	`0xE010_0000`–`0xFFFF_FFFF`	Vendor system space

Two of those rows explain things you have already used. The **peripheral** region starting at `0x4000_0000` is why the RCC registers from Week 03 were at `0x4002_1000`. And the **Cortex-M peripherals** region at `0xE000_0000` is where the NVIC lives, which is why `NVIC_ClearPendingIRQ` from Week 05 is defined by CMSIS-CORE rather than by ST: it is part of the core, identical on every Cortex-M, not part of the STM32.

5.1 Where your actual memory sits

Book board vs your board

	Book: STM32F091RC	Yours: STM32F303VCT6
Flash size	256 KB	256 KB
Flash range	`0x0800_0000`–`0x0803_FFFF`	`0x0800_0000`–`0x0803_FFFF`
SRAM size	32 KB	**40 KB**
SRAM range	`0x2000_0000`–`0x2000_7FFF`	`0x2000_0000`–`0x2000_9FFF`

Check the arithmetic yourself, it is a good exam-style exercise. 256 KB is $256 \times 1024 = 262144 =$ `0x40000` bytes, so a region starting at `0x0800_0000` ends at `0x0800_0000` + `0x3FFFF` = `0x0803_FFFF`. And 40 KB is $40 \times 1024 = 40960 =$ `0xA000`, so your SRAM ends at `0x2000_9FFF` rather than the book's `0x2000_7FFF`.

Note flash appears at `0x0800_0000`, inside the Code region but not at zero. Address `0x0000_0000` is an alias that can be mapped to flash, system memory, or SRAM depending on the boot pins, which is how the built-in bootloader works.

In the lab

Your Lab 10 Section 10.2.2 covers stack and heap on the STM32F3 specifically, and this is the map it is talking about. Both live in that 40 KB SRAM region, and they grow *towards each other*: the stack from the top downwards, the heap from the bottom upwards. When they meet, you have a collision, and it does not announce itself, it just corrupts whichever one got there second.

Lab 10 Task 2 asks you to write a heap allocator using **direct SRAM addressing**, defining a heap region by absolute address. Everything you need to do that safely is in the table above: you must place your region inside `0x2000_0000`–`0x2000_9FFF`, and you must keep it clear of wherever the linker put your globals and your stack. Read the `.map` file your build produces to find out where those actually are, rather than guessing.

And recall the constraint from Week 01: 40 KB total. The manual makes the point directly, that because RAM is measured in kilobytes on STM32, preventing leaks is essential to reliability. A leak on a desktop is untidy. A leak in 40 KB is a crash on a schedule.

6 Deep dive 5: Endianness

Endianness

The order in which the bytes of a multi-byte value are stored across consecutive addresses.

Little-endian: the **least**-significant byte goes at the **lowest** address.
Big-endian: the **most**-significant byte goes at the lowest address.

Take the 32-bit value in a register as four bytes B3 B2 B1 B0, with B0 least significant, stored at addresses A through A+3:

Address	Little-endian	Big-endian
A	B0 (least significant)	B3 (most significant)
A+1	B1	B2
A+2	B2	B1
A+3	B3 (most significant)	B0 (least significant)

For ARMv6-M, **instructions are always little-endian**. Data may be either, decided by the implementation. **STM32 microcontrollers are little-endian for data**, and for instructions of course.

Beware the trap

Endianness is invisible until you move bytes between systems, and then it is the whole problem. Two places it will find you this semester.

Sensors. Many I^2C and SPI sensors transmit **big-endian**, most-significant byte first. Your MCU is little-endian. So reading a 16-bit gyro rate as two bytes and casting the pair to an `int16_t` gives you a byte-swapped, wildly wrong number. You must assemble it explicitly: `value = ((int16_t)high « 8) | low;`. Your Lab 08 and Lab 09 both hit this, and it is the reason a sensor reading sometimes looks like noise rather than being subtly off.

Structs over a wire. Never `memcpy` a struct into a UART buffer and expect the other end to interpret it. Serialise field by field, in a documented order.

7 Deep dive 6: The stack

Stack, push, pop

> A **stack** is a last-in, first-out data structure that lets a program allocate a memory location, use it, and free it for later reuse, rather than permanently dedicating storage to a temporary.
>
> **Push**: write a data item to the next free stack location and update SP.
> **Pop**: read the item from the top of the stack and update SP.
>
> Push X then push Y, and the first pop returns **Y**, the second returns X.

Now the mechanics, and there are three facts that are easy to get backwards.

Fact one: SP points at the last item, not the first free space. So the value at address `[SP]` is live data.

Fact two: the stack grows towards *smaller* addresses. Pushing *decreases* SP by the number of bytes added. Popping increases it. ARM calls this a **full-descending** stack: full because SP points at occupied space, descending because it grows downwards.

Fact three: on Cortex-M every push and pop is 32 bits. No other size is possible. Consequently every legal SP value is a multiple of four, and the hardware hard-wires the bottom two bits of SP to zero.

Here is the book's walkthrough. Start with one item D at `0x2000_0010`:

Address	Initially	After push X	After push Y
`0x2000_0008`	free	free	**Y** ← SP
`0x2000_000C`	free	**X** ← SP	X
`0x2000_0010`	**D** ← SP	D	D

Pop once and you get Y with SP back at `0x2000_000C`. Pop again and you get X.

7.1 PUSH and POP as instructions

`PUSH` can take R0–R7 and `LR`. `POP` can take R0–R7 and `PC`. Each register moved adjusts SP by four.

The ordering rule is worth memorising because it looks arbitrary and is not: **regardless of how you write the operand list**, registers are pushed largest-number-first to the largest address, and popped smallest-number-first from the smallest address. So the same register always lands at the same relative offset, which is what makes the stack frame layout predictable.

- `PUSH {R0, R5-R7}` pushes R0, R5, R6, R7 and subtracts **16** from SP.
- `POP {R2, R4}` loads R2 and R4 and adds 8 to SP.

Note the asymmetry in what each can touch: `PUSH` accepts `LR`, `POP` accepts `PC`. That is not an accident, and the next section explains why.

8 Deep dive 7: Instructions, and how a subroutine call works

Machine vs assembly language

Machine language: each instruction is a numerical value stored in program memory, decoded directly by the CPU.
Assembly language: a human-readable form using **mnemonics** for the operation and its operands. An **assembler** translates one into the other.

Convention: the *first* operand is usually the destination, and will be overwritten with the result. The rest are sources.

The instruction set divides into five categories. You do not need to memorise every mnemonic, but you should recognise the shape:

Category	Type	Mnemonics
Data movement	Move	`MOV, MOVS, MRS, MSR`
Data processing	Math	`ADD, ADDS, ADCS, ADR, MULS, RSBS, SBCS, SUB, SUBS`
	Logic	`ANDS, EORS, ORRS, BICS, MVNS, TST`
	Compare	`CMP, CMN`
	Shift/rotate	`ASRS, LSLS, LSRS, RORS`
	Extend	`SXTB, SXTH, UXTB, UXTH`
	Reverse	`REV, REV16, REVSH`
Memory access	Load	`LDR, LDRB, LDRH, LDRSH, LDRSB, LDM`
	Store	`STR, STRB, STRH, STM`
	Stack	`PUSH, POP`
Control flow	Branch	`B, BL, BX, BLX,` and conditional forms `BEQ, BNE, BCS, BCC, BMI, BPL, BVS, BVC, BHI, BLS, BGE, BLT, BGT, BLE`
Miscellaneous	—	`BKPT, CPSID, CPSIE, WFE, WFI, SVC, YIELD, DMB, DSB, ISB, SEV, NOP`

Two details in that table repay attention.

The `S` suffix. Many instructions come in two versions: with `S` they update the condition flags in APSR, without `S` they do not. So `ADDS` sets N, Z, C, V based on the result, and `ADD` leaves them alone. That is what lets a comparison and a conditional branch be separate instructions: `CMP` sets the flags, then `BEQ` reads them.

`REV` and `REV16`. Byte-reversal instructions, and now you know exactly what they are for. Recall Section 5: these are the hardware answer to endianness conversion, one instruction instead of a shift-and-OR sequence.

Return address, and why LR exists

The **return address** is the address of the next instruction to execute after a subroutine completes.

> `BL Subroutine1` branches to the subroutine *and saves the return address in LR*. `BLX R0` does the same for an address held in a register.
>
> Returning means copying the return address into PC, which is `BX LR`.

Now the important consequence, which explains the `PUSH/POP` asymmetry from Section 6.

There is only *one* LR. So if subroutine `Sub_1` calls `Sub_2`, the `BL` into `Sub_2` overwrites LR, destroying `Sub_1`'s own return address. `Sub_1` must therefore **save LR onto the stack before calling anything**, and it returns by popping the saved value straight into PC with `POP {PC}`.

Which is why `PUSH` accepts LR and `POP` accepts PC. They are two halves of one idiom:

Listing 2: The universal shape of a non-leaf function.

```
Sub_1:
        PUSH  {R4, LR}          ; save LR because we are about to call out
        ...
        BL    Sub_2             ; this overwrites LR
        ...
        POP   {R4, PC}          ; pop the saved LR directly into PC = return
```

Beware the trap

> A **leaf** function, one that calls nothing, does not need to save LR and can return with `BX LR`. A **non-leaf** function must save it. Forget, and the function returns to wherever the most recent nested call came from, producing a crash whose stack trace makes no sense at all.
>
> The compiler handles this for you in C. It matters because it is exactly what you will be reading in week 10 when you look at disassembly, and because `PUSH {..., LR}` at the top of a function is the fastest way to tell at a glance whether that function calls anything.

8.1 Load and store multiple

`LDM` loads a list of registers from consecutive memory starting at a given address, and `STM` stores a list. One instruction moving many words, which reduces both code size and time. `PUSH` and `POP` are, in effect, specialisations of these with SP as the address and the update built in.

And that is enough machinery. Week 7 uses every bit of it.

9 Tying it back

We opened the CPU so that week 7 makes sense, and here is what we found.

The core is a program counter, a small fast register file, an ALU, slower larger data memory, and control logic that decodes instructions into signals. Normally PC advances by one instruction, and **a branch, a subroutine call, or an interrupt** is simply something that changes PC differently, which is the sentence to keep hold of going into next week.

ARMv6-M commits to **load/store**, so memory data must come into a register to be processed, which is why the read-modify-write from Week 03 is genuinely three transactions and genuinely interruptible. It commits to a **32-bit native type**, which is why `uint32_t` accesses are atomic and `uint64_t` accesses are not, and why your M4's hardware FPU matters so much for the PID controller in your project.

There are sixteen core registers, with R13 as **SP**, R14 as **LR**, R15 as **PC**, plus PSR in its three views, PRIMASK, and CONTROL. Memory is a byte-addressable 4 GB space divided into fixed regions, with your flash at `0x0800_0000` and your 40 KB of SRAM at `0x2000_0000` through `0x2000_9FFF`, which is the arena your Lab 10 heap allocator has to live inside. Data is **little-endian**, which is why sensor readings need explicit byte assembly. The stack is **full-descending**,

32 bits at a time, with SP pointing at live data. And **LR** holds the return address, which is why any function that calls another must push LR and pop it into PC.

Next week we use all of it at once. When an interrupt arrives, the processor pushes a specific set of registers onto a specific stack, loads PC from a vector table, runs your handler, and unwinds. Every noun in that sentence is now defined.

Cheat sheet: Week 06

The registers. Learn this table.

R0–R12 general purpose **R13** = **SP** stack pointer **R14** = **LR** link register, holds return address **R15** = **PC** program counter

Two stack pointers exist: **MSP** (main) and **PSP** (process). Simple apps use MSP only; kernels use both.

PSR, one register, three views: **APSR** = flags N, Z, C, V · **IPSR** = current exception number · **EPSR** = Thumb mode bit.
PRIMASK = global interrupt disable. **CONTROL.SPSEL** = picks MSP or PSP in Thread mode.
M4 extras: **BASEPRI** (mask below a priority), **FAULTMASK**, **S0–S31** + `FPSCR` for the FPU.

Architecture commitments

Load/store: only register data can be processed. Memory needs `LDR` → operate → `STR`. Three instructions, and interruptible in the middle. *This is why `BSRR` exists.*
Native type = 32-bit integer. `uint32_t` access is one instruction (atomic); `uint64_t` is two (not atomic).
M0 = ARMv6-M, von Neumann, no FPU, no DSP. M4 = ARMv7E-M, Harvard, single-precision FPU, DSP.
On your M4 use `float` and `2.0f` and `sinf()`. `double` falls back to software.

Memory map, byte-addressable, 2^{32} = 4 GB

`0x0000_0000` Code `0x2000_0000` SRAM `0x4000_0000` Peripheral `0x6000_0000` External RAM
`0xA000_0000` External device `0xE000_0000` Cortex-M peripherals (NVIC, SysTick) `0xE010_0000` System

Your F303VCT6: 256 KB flash at `0x0800_0000`–`0x0803_FFFF`, **40 KB** SRAM at `0x2000_0000`–`0x2000_9FFF`.
Book's F091RC: same flash, **32 KB** SRAM ending `0x2000_7FFF`.
Stack grows down from the top of SRAM, heap grows up from the bottom. They collide silently.

Endianness

Little-endian: least-significant byte at the lowest address. **STM32 is little-endian**, data and instructions.
Instructions on ARMv6-M are *always* little-endian regardless of the data setting.
Many SPI and I^2C sensors send **big-endian**. Assemble explicitly: `v = ((int16_t)hi « 8) | lo;` Do not cast a byte pair. `REV/REV16` are the hardware byte-swap instructions.

The stack, three facts people get backwards

1. **SP points at the last item**, not the first free slot.
2. **It grows towards smaller addresses.** Push decreases SP, pop increases it. ARM calls this **full-descending**.
3. **Every push and pop is 32 bits**, no exceptions, so SP is always a multiple of 4 and its bottom two bits are hard-wired zero.

`PUSH` takes R0–R7 and **LR**. `POP` takes R0–R7 and **PC**. Order is fixed regardless of how you write the list: highest register to highest address.
`PUSH {R0, R5-R7}` subtracts 16 from SP.

Subroutine call and return

`BL target` → branch and **save return address in LR**. `BX LR` → return.
There is only one LR, so a **non-leaf** function must `PUSH {..., LR}` on entry and `POP {..., PC}` on exit. A **leaf** function can just `BX LR`.
`PUSH {..., LR}` at the top of a disassembled function means it calls something.

The `S` **suffix** (`ADDS` vs `ADD`) means "update the APSR flags". That is how `CMP` then `BEQ` works as two instructions.

Likely exam prompts from this chapter

· Name R13, R14, R15 and state each one's function. *Near certain.*
· Name the three views of the PSR and what each holds.
· Given a start address and a size in KB, compute the end address of a memory region.
· Draw the stack before and after a given sequence of pushes and pops, showing SP.
· State how much SP changes for `PUSH {R0, R4-R6}` and in which direction.
· Explain what load/store architecture means and give one consequence.
· Show how a 32-bit value is laid out in little-endian vs big-endian memory.
· Why must a subroutine that calls another subroutine save LR?